JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Interface TemporalField

**All Known Implementing Classes:**

ChronoField

---

`public interface` **`TemporalField`**

A field of date-time, such as month-of-year or minute-of-hour.

Date and time is expressed using fields which partition the time-line into something meaningful for humans. Implementations of this interface represent those fields.

The most commonly used units are defined in `ChronoField`. Further fields are supplied in `IsoFields`, `WeekFields` and `JulianFields`. Fields can also be written by application code by implementing this interface.

The field works using double dispatch. Client code calls methods on a date-time like `LocalDateTime` which check if the field is a `ChronoField`. If it is, then the date-time must handle it. Otherwise, the method call is re-dispatched to the matching method in this interface.

**Implementation Requirements:**

This interface must be implemented with care to ensure other classes operate correctly. All implementations that can be instantiated must be final, immutable and thread-safe. Implementations should be `Serializable` where possible. An enum is as effective implementation choice.

**Since:**

1.8

## *Method Summary*

| All Methods | Instance Methods | |
|---|---|---|
| **Modifier and Type** | **Method** | **Description** |
| `<R extends Temporal> R` | **`adjustInto`**`(R temporal, long newValue)` | Returns a copy of the specified temporal object with the value of this field set. |
| `TemporalUnit` | **`getBaseUnit`**`()` | Gets the unit that the field is measured in. |
| `default String` | **`getDisplayName`**`(Locale locale)` | Gets the display name for the field in the requested locale. |

| | | |
|---|---|---|
| long | **getFrom**(TemporalAccessor tempor | Gets the value of this field from the specified temporal object. |
| TemporalUnit | **getRangeUnit**() | Gets the range that the field is bound by. |
| boolean | **isDateBased**() | Checks if this field represents a component of a date. |
| boolean | **isSupportedBy**(TemporalAccessor tempor | Checks if this field is supported by the temporal object. |
| boolean | **isTimeBased**() | Checks if this field represents a component of a time. |
| ValueRange | **range**() | Gets the range of valid values for the field. |
| ValueRange | **rangeRefinedBy**(TemporalAccessor tempor | Get the range of valid values for this field using the temporal object to refine the result. |
| default TemporalAccessor | **resolve**(Map<TemporalField, Long> fieldValues, TemporalAccessor partial ResolverStyle resolverSt | Resolves this field to provide a simpler alternative or a date. |
| String | **toString**() | Gets a descriptive name for the field. |

## *Method Details*

### getDisplayName

```
default String getDisplayName(Locale locale)
```

Gets the display name for the field in the requested locale.

If there is no display name for the locale then a suitable default must be returned.

The default implementation must check the locale is not null and return `toString()`.

**Parameters:**

`locale` - the locale to use, not null

**Returns:**

the display name for the locale or a suitable default, not null

## getBaseUnit

`TemporalUnit getBaseUnit()`

Gets the unit that the field is measured in.

The unit of the field is the period that varies within the range. For example, in the field 'MonthOfYear', the unit is 'Months'. See also `getRangeUnit()`.

**Returns:**

the unit defining the base unit of the field, not null

## getRangeUnit

`TemporalUnit getRangeUnit()`

Gets the range that the field is bound by.

The range of the field is the period that the field varies within. For example, in the field 'MonthOfYear', the range is 'Years'. See also `getBaseUnit()`.

The range is never null. For example, the 'Year' field is shorthand for 'YearOfForever'. It therefore has a unit of 'Years' and a range of 'Forever'.

**Returns:**

the unit defining the range of the field, not null

## range

`ValueRange range()`

Gets the range of valid values for the field.

All fields can be expressed as a `long` integer. This method returns an object that describes the valid range for that value. This method is generally only applicable to the ISO-8601 calendar system.

Note that the result only describes the minimum and maximum valid values and it is important not to read too much into them. For example, there could be values within the range that are invalid for the field.

**Returns:**

the range of valid values for the field, not null

## isDateBased

```
boolean isDateBased()
```

Checks if this field represents a component of a date.

A field is date-based if it can be derived from `EPOCH_DAY`. Note that it is valid for both `isDateBased()` and `isTimeBased()` to return false, such as when representing a field like minute-of-week.

**Returns:**

true if this field is a component of a date

## isTimeBased

```
boolean isTimeBased()
```

Checks if this field represents a component of a time.

A field is time-based if it can be derived from `NANO_OF_DAY`. Note that it is valid for both `isDateBased()` and `isTimeBased()` to return false, such as when representing a field like minute-of-week.

**Returns:**

true if this field is a component of a time

## isSupportedBy

```
boolean isSupportedBy(TemporalAccessor temporal)
```

Checks if this field is supported by the temporal object.

This determines whether the temporal accessor supports this field. If this returns false, then the temporal cannot be queried for this field.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `TemporalAccessor.isSupported(TemporalField)`:

```
// these two lines are equivalent, but the second approach is recommended
supported = thisField.isSupportedBy(temporal);
supported = temporal.isSupported(thisField);
```

It is recommended to use the second approach, `isSupported(thisField)`, as it is a lot clearer to read in code.

Implementations should determine whether they are supported using the fields available in `ChronoField`.

**Parameters:**

`temporal` - the temporal object to query, not null

**Returns:**

true if the date-time can be queried for this field, false if not

## rangeRefinedBy

`ValueRange rangeRefinedBy(TemporalAccessor temporal)`

Get the range of valid values for this field using the temporal object to refine the result.

This uses the temporal object to find the range of valid values for the field. This is similar to `range()`, however this method refines the result using the temporal. For example, if the field is `DAY_OF_MONTH` the `range` method is not accurate as there are four possible month lengths, 28, 29, 30 and 31 days. Using this method with a date allows the range to be accurate, returning just one of those four options.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `TemporalAccessor.range(TemporalField)`:

```
// these two lines are equivalent, but the second approach is recommended
range = thisField.rangeRefinedBy(temporal);
range = temporal.range(thisField);
```

It is recommended to use the second approach, `range(thisField)`, as it is a lot clearer to read in code.

Implementations should perform any queries or calculations using the fields available in `ChronoField`. If the field is not supported an `UnsupportedTemporalTypeException` must be thrown.

**Parameters:**

`temporal` - the temporal object used to refine the result, not null

**Returns:**

the range of valid values for this field, not null

**Throws:**

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported by the temporal

## getFrom

`long getFrom(TemporalAccessor temporal)`

Gets the value of this field from the specified temporal object.

This queries the temporal object for the value of this field.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `TemporalAccessor.getLong(TemporalField)` (or `TemporalAccessor.get(TemporalField)`):

```
// these two lines are equivalent, but the second approach is recommended
value = thisField.getFrom(temporal);
value = temporal.getLong(thisField);
```

It is recommended to use the second approach, `getLong(thisField)`, as it is a lot clearer to read in code.

Implementations should perform any queries or calculations using the fields available in `ChronoField`. If the field is not supported an `UnsupportedTemporalTypeException` must be thrown.

**Parameters:**

`temporal` - the temporal object to query, not null

**Returns:**

the value of this field, not null

**Throws:**

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported by the temporal

`ArithmeticException` - if numeric overflow occurs

## adjustInto

```
<R extends Temporal> R adjustInto(R temporal,
                                  long newValue)
```

Returns
a copy of the specified temporal object with the value of this field set.

This returns a new temporal object based on the specified one with the value for this field changed. For example, on a `LocalDate`, this could be used to set the year, month or day-of-month. The returned object has the same observable type as the specified object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then changing the month to February would be unclear. In cases like this, the implementation is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `Temporal.with(TemporalField, long)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisField.adjustInto(temporal, newValue);
temporal = temporal.with(thisField, newValue);
```

It is recommended to use the second approach, `with(thisField, newValue)`, as it is a lot clearer to read in code.

Implementations should perform any queries or calculations using the fields available in `ChronoField`. If the field is not supported an `UnsupportedTemporalTypeException` must be thrown.

Implementations must not alter the specified temporal object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

**Type Parameters:**

`R` - the type of the Temporal object

**Parameters:**

`temporal` - the temporal object to adjust, not null

`newValue` - the new value of the field

**Returns:**

the adjusted temporal object, not null

**Throws:**

DateTimeException - if the field cannot be set

UnsupportedTemporalTypeException - if the field is not supported by the temporal

ArithmeticException - if numeric overflow occurs

## resolve

```
default TemporalAccessor resolve
(Map<TemporalField, Long> fieldValues,
 TemporalAccessor partialTemporal,
 ResolverStyle resolverStyle)
```

Resolves this field to provide a simpler alternative or a date.

This method is invoked during the resolve phase of parsing. It is designed to allow application defined fields to be simplified into more standard fields, such as those on `ChronoField`, or into a date.

Applications should not normally invoke this method directly.

**Implementation Requirements:**

If an implementation represents a field that can be simplified, or combined with others, then this method must be implemented.

The specified map contains the current state of the parse. The map is mutable and must be mutated to resolve the field and any related fields. This method will only be invoked during parsing if the map contains this field, and implementations should therefore assume this field is present.

Resolving a field will consist of looking at the value of this field, and potentially other fields, and either updating the map with a simpler value, such as a `ChronoField`, or returning a complete `ChronoLocalDate`. If a resolve is successful, the code must remove all the fields that were resolved from the map, including this field.

For example, the `IsoFields` class contains the quarter-of-year and day-of-quarter fields. The implementation of this method in that class resolves the two fields plus the `YEAR` into a complete `LocalDate`. The resolve method will remove all three fields from the map before returning the `LocalDate`.

A partially complete temporal is used to allow the chronology and zone to be queried. In general, only the chronology will be needed. Querying items other than the zone or chronology is undefined and must not be relied on. The behavior of other methods such as `get`, `getLong`, `range` and `isSupported` is unpredictable and the results undefined.

If resolution should be possible, but the data is invalid, the resolver style should be used to determine an appropriate level of leniency, which may require throwing a `DateTimeException` or `ArithmeticException`. If no resolution is possible, the resolve method must return null.

When resolving time fields, the map will be altered and null returned. When resolving date fields, the date is normally returned from the method, with the map altered to remove the resolved fields. However, it would also be acceptable for the date fields to be resolved into other `ChronoField` instances that can produce a date, such as `EPOCH_DAY`.

Not all `TemporalAccessor` implementations are accepted as return values. Implementations that call this method must accept `ChronoLocalDate`, `ChronoLocalDateTime`, `ChronoZonedDateTime` and `LocalTime`.

The default implementation must return null.

**Parameters:**

`fieldValues` - the map of fields to values, which can be updated, not null

`partialTemporal` - the partially complete temporal to query for zone and chronology; querying for other things is undefined and not recommended, not null

`resolverStyle` - the requested type of resolve, not null

**Returns:**

the resolved temporal object; null if resolving only changed the map, or no resolve occurred

**Throws:**

`ArithmeticException` - if numeric overflow occurs

`DateTimeException` - if resolving results in an error. This must not be thrown by querying a field on the temporal without first checking if it is supported

## toString

`String toString()`

Gets a descriptive name for the field.

The should be of the format 'BaseOfRange', such as 'MonthOfYear', unless the field has a range of `FOREVER`, when only the base unit is mentioned, such as 'Year' or 'Era'.

**Overrides:**

`toString` in class `Object`

**Returns:**

the name of the field, not null

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.